HTTPv2 The docking protocol interface document of facial recognition terminal

HTTPv2 The docking protocol interface document of facial recognition terminal
- Update Record

HTTP Protocol Description

HTTP API interface
- Call process
- Regarding HTTP Connection Maintenance
- Exception handling
  - Network abnormality
  - The server refused the connection

Connect to keep alive
- API Device Heartbeat Active Package
  - Request Parameters
  - Parameter description of return value
  - Return error code

Device operating parameters
- API - Upload current device parameters
- API-The device actively obtains working parameters

Remote control
- API-Remote operation command
  - Example of Request Parameters
  - Return value parameter description
  - RegisterIdentifyTicket Registration certificate object structure in command
  - RegisterType Registration Type
  - PushSoftware The object structure of firmware files in commands
  - PushSystemFile System file object structure in commands
  - FileType File Type
- API-Feedback on the results of personnel registration credentials
  - Example of Request Parameters
  - Parameter description of Return value
- API-Upload device camera snapshot
  - Parameter description of return value
- API-Obtain Personnel Authorization Information
  - Request parameters
  - Example of Request Parameters
  - Return value parameter description
  - PeopleJson Personnel data format
    - Holidays Holidays
  - Return parameter example
- API-Feedback to obtain personnel storage results
- API-Feedback on the operation results of deleting personnel
- API-Push Personnel Information

Online Authentication
- API-Request server authentication
  - RecordDetail Field Description
  - RecordDetail Parameter Examples
  - Parameter Examples
  - Return Example
  - Request content compression

Success returns results
Record management
API-Upload punch-in records
RecordDetail Field Description
RecordType Event type
RecordDetail Parameter examples
** Request Example**
Return error code
Request content compression
API-Upload System Records
Record Field Description
RecordType Field Description
RecordDetail Parameter examples
Parameter examples
Return Example
Return error code

- **Communication protocol based on HTTP**
- **Document version 6.0**
- *Release date: June 12, 2024**
- **Modified by Lai Jinjie**

Update Record

Version Number	Modified Date	Explanation

HTTP Protocol Description

- **Supports HTTP 1.1
- **GZIP compression technology**
- **Supports TLS1.2 and TLS1.3**

HTTP API interface

Call process

-The device send a heartbeat packet to the server first, and the server responds with an OK message

- Device sends working parameters to the service
- After the server responds to the heartbeat protection package, it decides the next request to be initiated based on the response content
- **Priority syncParameter > Remote > deletePeople > addPeople**
- After processing the operations required by the server, check for any accumulated un uploaded records

- If there are accumulated records that have not been uploaded, push the records to the server
- The device needs to ensure that the interval between two heartbeat keep alive packets does not exceed the set polling interval.
- The heartbeat interval is 15 seconds. If the first heartbeat packet is sent at 10:00:00 and there is a request to delete or add personnel in the server response, the delete or add personnel interface will be called repeatedly. However, it is necessary to ensure that another heartbeat is sent at 10:00:15.
    - **Processing parameter synchronization, responding to remote operations, deleting personnel, adding personnel, and pushing records between two heartbeat intervals.**

Regarding HTTP Connection Maintenance

The device and server should first establish a connection through a keep alive packet. After the server responds successfully, it should maintain a long HTTP connection until all operations are completed before disconnecting. If there are no operations to be performed, the connection can be immediately disconnected until the next keep alive packet

- When the server supports the HTTP2.0 protocol, priority should be given to using HTTP2.0 for requests
- One device should ensure that it only maintains one HTTP connection with the server to avoid multiple concurrent connections

Exception handling

Network abnormality

- 1.Unable to connect to the server upon startup, failed to send parameter upload request
    - Record parameter upload status as' not uploaded ', communication enters sleep mode, waiting for the next interval time
    - When the interval time is up, send heartbeat packets. If the server does not respond, continue to sleep and wait
    - After the heartbeat packet responds, check the parameter upload status. If it is not successfully sent, send it again, and then continue to respond to the subsequent operations requested by the server (refer to the calling process)
- 2.The device cannot connect to the server during operation
    - Any request should be accompanied by a timeout and response retry - it is recommended to set a timeout of 2 seconds and retry the request 3 times
      After the request fails, enter communication sleep mode and wait for the interval between sending the keep alive packet to arrive
    - Follow up process reference question 1

The server refused the connection

- After the device sends a punch-in record to the server API URL request, the response returned by the server
- If the success field returns a value of 401/403, it indicates no permission. The device should enter request silence and wait for the next keep alive cycle. After the keep alive cycle, send the keep alive first

- When the Status status in the heap is 401/403/400, etc., it is also necessary to wait for the next keep alive packet cycle before sending a keep alive packet to detect the server status.
- Continue executing other API calls only when the server returns an HTTP status of 200 and a Success field of 1

Connect to keep alive

API Device Heartbeat Active Package

Brief description:

- When the device is idle, send a keep alive packet more than [keep alive interval] seconds after the last communication with the server to confirm if there are any unfinished tasks

Request URL:

- http://Server IP:port**/Device/Keepalive**

Request method:

- POST

Content-Type

- application/json

Request Parameters

Field	Type	Explain
SN	string	Device SN
RelayStatus	int	Relay state 0- indicates COM and NC are normally closed 1- indicates COM and NO are normally closed
KeepOpenStatus	int	Normally open 0- indicates normally closed 1- indicates normally open
DoorSensorStatus	int	Door sensor status 0- indicates closed 1- indicates open
LockDoorStatus	int	Door locked status 0- indicates unlocked 1- indicates locked

Field	Type	Explain
AlarmStatus	string	Door alarm status An empty string indicates no alarm, otherwise there will be a specific alarm name fire -- fire alarm blacklist -- blacklist alarm anti -- tamper alarm illegal -- unauthorized verification password -- force alarm password openTimeout -- door timeout alarm doorSensor -- door sensor alarm When there are multiple alarms, use commas to separate it, such as fire, blacklist

Example of Request Parameters

```
{
    "SN":"FC-8200H12345678",
    "RelayStatus":  0,
    "KeepOpenStatus":   0,
    "DoorSensorStatus": 0,
    "LockDoorStatus":   0,
    "AlarmStatus":  "",
}
```

Parameter description of return value

Parameter Name	Type	Necessary	Description
Success	int	Yes	1: Success< 401/403 indicates that the server has been connected, but subsequent connections are denied due to the SN not being registered or installed
AddPeople	int	No	>0- indicates that there is a need to add personnel to the device After receiving the device, a request for People/DownloadPeopleList needs to be initiated =0 or when there is no such field, it indicates that no processing is required
DeletePeople	int	No	>Indicates the need to delete personnel from the device After the device receives it, it needs to initiate a request for People/DownloadPeopleList

Parameter Name	Type	Necessary	Description
SyncParameter	int	No	1- Indicates that there are parameters that need to be set to the device After the device receives them, it needs to initiate a Device/DownloadWorkSetting request
Remote	int	No	1- Indicates that there is a remote operation that needs to be processed. After the device receives it, it needs to initiate a Device/RemoteCommand request
UploadWorkParameter	int	No	1- Indicates that the device is required to upload its operating parameters< After receiving the device, it needs to initiate a Device/UploadWorkSetting request

Example of return value

```
{
    "Success":1,
    "AddPeople":1,
    "DeletePeople":1,
    "SyncParameter":1,
    "Remote":1,
    "UploadworkParameter": 1
}

{
    "Success":1
}
```

- **Priority UploadWorkParameter >Remote > SyncParameter> DeletePeople> AddPeople**

Return error code

```
{
    "Success":401   //When returning non-zero values such as 401 and 400. If the device encounters a problem on the platform, it will no longer send punch in records and device parameters to the server. But it will still periodically send keep alive packages
}
/*
When testing the connection of the device again, prompt the text based on the return value
(1) 401 prompt
① Chinese: Connected to the server, but the device is not activated
②English: Connected to server, but device is not activated
(2) 400 prompt
① Chinese: Connected to server, but SN does not comply with platform rules
```

```
② Chinese: Connected to server, but SN does not comply with platform rules
(3) 0 prompt
① Chinese: Connected to server, testing completed
②English: Connected to server, testing completed
(4) Other prompts based on message content
*/
```

Device operating parameters

Please refer to the document [Device Operating Parameters. md] (./Device Operating Parameters. md)

API - Upload current device parameters

Brief description:

-Send once when the device is turned on
-When the device properties change, they will be uploaded again

Request URL:

- http://Server IP:port**/Device/UploadWorkSetting**

Request method:

- POST

Content-Type

- application/json

Request parameters:

- **Example of Request Parameters**


```
{
       "DeviceSN": "FC-8200H12345675",
        "FireAlarm": 0,
        "DoorLongOpenAlarm": 0,
        "DoorLongOpenTime": 0,
        "DoorSensorAlarm": 0,
        "DoorSensorAlarmTimegroup": {
            "Week1": "",
            "Week2": "",
            "Week3": "",
            "Week4": "",
            "Week5": "",
            "Week6": "",
            "Week7": ""
        },
        "BlacklistAlarm": 1,
        "AntiDisassemblyAlarm": 1,
        "IllegalVerificationAlarm": 0,
        "IllegalVerificationAlarmLimit": 30,
        "UseUserCloseAlarm": 1,
        "PasswordAlarm": 0,
        "PasswordAlarm_Password": "",
```

```
        "PasswordAlarm_Mode": 1,
        "AlarmClocks": [
            {
                "Num": 1,
                "Clock": "00:00",
                "Times": 0
            },
            {
                "Num": 24,
                "Clock": "00:00",
                "Times": 0
            }
        ],
        "UseBodyTemperature": 1,
        "UseFahrenheitDisplay": 1,
        "TemperatureCompensate": 0,
        "TemperatureAlarmThreshold": 37.29999,
        "TemperatureDisplay": 1,
        "CardBytes": 3,
        "AccessType": 0,
        "WgFormat": 26,
        "WGContent": 1,
        "ReleaseTime": 3,
        "FreeOpen": 0,
        "OpenInterval": 2,
        "OpenInterval_SaveRecord": 0,
        "Relay": 0,
        "ShortMessage": "",
        "VerificationType": 1,
        "OverdueRemind": 1,
        "OverdueRemind_Day": 3,
        "TimingOpen": 0,
        "TimingOpen_mode": 1,
        "TimingOpen_timegroup": {
            "Week1": "",
            "Week2": "",
            "Week3": "",
            "Week4": "",
            "Week5": "",
            "Week6": "",
            "Week7": ""
        },
        "TimingLocked": 0,
        "TimingLocked_timegroup": {
            "Week1": "",
            "Week2": "",
            "Week3": "",
            "Week4": "",
            "Week5": "",
            "Week6": "",
            "Week7": ""
        },
        "VisitorRootPassword": "",
        "MultiPerson": 1,
        "UseElevator": 0,
```

```
        "ElevatorPorts": [
            {
                "Num": 1,
                "RelayType": 2,
                "ReleaseTime": 2,
                "TimingOpen": {
                    "Use": 0,
                    "Open": 3,
                    "Timegroup": {
                        "Week1": "",
                        "Week2": "",
                        "Week3": "",
                        "Week4": "",
                        "Week5": "",
                        "Week6": "",
                        "Week7": ""
                    }
                }
            },
            {
                "Num": 64,
                "RelayType": 2,
                "ReleaseTime": 2,
                "TimingOpen": {
                    "Use": 0,
                    "Open": 3,
                    "Timegroup": {
                        "Week1": "",
                        "Week7": ""
                    }
                }
            }
        ],
        "FaceIR": 1,
        "FaceIRThreshold": 5,
        "FaceDistance": 3,
        "FaceThreshold": 58,
        "FPComparison": 80,
        "FaceMask": 0,
        "FaceMaskThreshold": 65,
        "Holidays": [],
        "Language": 1,
        "SystemTime": 1718421632,
        "UseNTP": 1,
        "TimeZone": 8,
        "Volume": 6,
        "Voice": 1,
        "ConnectPassword": "",
        "UseWired": 1,
        "WiredDHCP": 0,
        "WiredIP": "192.168.1.103",
        "WiredIPMask": "255.255.255.0",
        "WiredGteway": "192.168.1.1",
        "WiredDNS": "192.168.1.1",
        "WiredMAC": "7E-87-16-C1-E6-51",
```

```
        "UseWIFI": 0,
        "WIFIDHCP": 0,
        "WIFIIP": "192.168.1.150",
        "WIFIIPMask": "255.255.255.0",
        "WIFIGteway": "192.168.1.1",
        "WIFIDNS": "192.168.1.1",
        "WIFIMAC": "34-7D-E4-2D-63-3B",
        "WIFIAPName": "",
        "WIFIAPPassword": "",
        "UseWebPage": 1,
        "HTTPPort": 80,
        "HTTPSPort": 443,
        "WebPageUseSSL": 1,
        "UseUDP": 1,
        "UDPPort": 8101,
        "UseTelnet": 1,
        "TelnetPort": 23,
        "UseRTSP": 1,
        "RTSPPort": 554,
        "RTSPUser": "admin",
        "RTSPPassword": "12345678",
        "UseTCPClient": 0,
        "UseUDPClient": 0,
        "ServerAddress": "47.92.31.75",
        "ServerPort": 9003,
        "KeepaliveTime": 30,
        "UseHTTPClient": 0,
        "HTTPClient_ProtocolType": 100,
        "HTTPClient_ServerAddr": "http://192.168.1.100",
        "HTTPClient_KeepaliveTime": 30,
        "HTTPClient_UseGZIP": 0,
        "UseMQTTClient": 0,
        "UseMQTTSSL": 0,
        "MQTTServerAddr": "192.168.1.100",
        "MQTTPort": 0,
        "MQTTLoginName": "",
        "MQTTLoginPassword": "",
        "MQTTPublishTopic": "",
        "MQTTSubscribeTopic": "",
        "MQTT_KeepaliveTime": 30,
        "MQTT_UseGZIP": 0,
        "UseWebsocketClient": 0,
        "WebsocketClient_ProtocolType": 100,
        "WebsocketClient_ServerAddr": "ws://192.168.1.100/ws",
        "WebsocketClient_UseGZIP": 0,
        "WebsocketClient_KeepaliveTime": 30,
        "UseYZW": 0,
        "YZWAddr": "http://192.168.1.10",
        "RunDays": 0,
        "FormatCount": 0,
        "WatchDogCount": 7,
        "BootTime": 1718418194,
        "RelayStatus": 0,
        "KeepOpenStatus": 0,
        "DoorSensorStatus": 0,
```

```
        "LockDoorStatus": 0,
        "AlarmStatus": "\"\"",
        "RecordAutoCycle": 0,
        "SaveUnregistered": 1,
        "SaveRecordPicture": 1,
        "PeopleStorageInfo": {
            "Person": {
                "Max": 20000,
                "Current": 1
            },
            "Face": {
                "Max": 20000,
                "Current": 1
            },
            "Card": {
                "Max": 20000,
                "Current": 0
            },
            "Fingerprint": {
                "Max": 0,
                "Current": 0
            },
            "PalmVein": {
                "Max": 10000,
                "Current": 0
            },
            "Pasword": {
                "Max": 20000,
                "Current": 0
            },
            "Admin": {
                "Max": 5,
                "Current": 0
            }
        },
        "RecordStorageInfo": {
            "VerifyRecord": {
                "Max": 1000000,
                "Current": 4
            },
            "DoorRecord": {
                "Max": 10000,
                "Current": 4
            },
            "SystemRecord": {
                "Max": 10000,
                "Current": 7
            },
            "RecordPhoto": {
                "Max": 10000,
                "Current": 4
            }
        },
        "DeviceSN": "FC-8190H24052799",
        "DeviceName": "",
```

```
        "FirmwareVerson": "8.46",
        "FingerprintVerson": "-",
        "FaceVerson": "6.01",
        "Manufacturer": "",
        "ManufacturerPhone": "",
        "Website": "",
        "ProductionDate": "2024-06-15",
        "OEMText": "",
        "AutoRestart": 0,
        "AutoRestartTime": "02:00",
        "TimeGroups": [
            {
                "Num": 1,
                "Week1": "00:00-23:59",
                "Week2": "00:00-23:59",
                "Week3": "00:00-23:59",
                "Week4": "00:00-23:59",
                "Week5": "00:00-23:59",
                "Week6": "00:00-23:59",
                "Week7": "00:00-23:59"
            },
            {
                "Num": 64,
                "Week1": "",
                "Week7": ""
            }
        ],
        "DisplayBrightness": 6,
        "MenuPassword": "0000",
        "ShowIR": 0,
        "ShowPersonPhoto": 1,
        "PlayPersonName": 1,
        "RecognitionButon": 0,
        "UnregisteredWarn": 0,
        "ShowPersonName": 1,
        "FillLight": 2,
        "FunctionList": {
            "BodyTemperature":  true,
            "Fingerprint":  false,
            "Palmvein": true,
            "Face": true,
            "QRCode":   true,
            "FaceMask": true,
            "SafetyHelmet": false,
            "Lift": true,
            "AlarmClock":   true,
            "ExcelFile":    false,
            "ZipFile":  false,
            "TimeGroup":    64,
            "WIFI": true,
            "HTTPClient_V1":    true,
            "HTTPClient_V2":    true,
            "MQTT": true,
            "YZW":  true,
            "Websocket_V1": true,
```

```
            "Websocket_V2": true
        }
}
```

Return parameter description

Parameter Name	Type	Nnecessary	Description
Success	int	Yes	1 indicates successful operation; when != 1, it is necessary to indicate the error description in the Message
Message	string	No	Error message description

Return value example

```
{
    "Success":1
}
```

API-The device actively obtains working parameters

Brief description:

When the device sends a heartbeat packet and the SyncParameter field in the server's response packet is set to 1, the device will request this interface

Request URL:

- http://Server IP:port**/Device/DownloadWorkSetting**

Request method:

- POST

Content-Type

- application/json

Request parameter description

Parameter Name	Type	Nnecessary	Description
SN	string	Yes	Device ID

Example of Request Parameters

```
{
    "SN":"FC-8200H12345678"
}
```

Return parameter description

Field	Type	Required	Explain

Field	Type	Required	Explain
Success	int	Yes	1-- indicates success Other are error codes, and the error content needs to be indicated in the Message
Message	string	No	Error message description, please refer to Chapter 3 for details
All Modified Parameters			

Return value example


```
{
       "Success": 1,
       "DeviceSN": "FC-8200H12345675",
        "FireAlarm": 0,
        "DoorLongOpenAlarm": 0,
        "DoorLongOpenTime": 0,
        "DoorSensorAlarm": 0,
        "DoorSensorAlarmTimegroup": {
            "Week1": "",
            "Week2": "",
            "Week3": "",
            "Week4": "",
            "Week5": "",
            "Week6": "",
            "Week7": ""
        },
        "BlacklistAlarm": 1,
        "AntiDisassemblyAlarm": 1,
        "IllegalVerificationAlarm": 0,
        "IllegalVerificationAlarmLimit": 30,
        "UseUserCloseAlarm": 1,
        "PasswordAlarm": 0,
        "PasswordAlarm_Password": "",
        "PasswordAlarm_Mode": 1,
        "AlarmClocks": [
            {
                "Num": 1,
                "Clock": "00:00",
                "Times": 0
            },
            {
                "Num": 24,
                "Clock": "00:00",
                "Times": 0
            }
        ],
        "UseBodyTemperature": 1,
        "UseFahrenheitDisplay": 1,
        "TemperatureCompensate": 0,
        "TemperatureAlarmThreshold": 37.29999,
```

```
        "TemperatureDisplay": 1,
        "CardBytes": 3,
        "AccessType": 0,
        "WgFormat": 26,
        "WGContent": 1,
        "ReleaseTime": 3,
        "FreeOpen": 0,
        "OpenInterval": 2,
        "OpenInterval_SaveRecord": 0,
        "Relay": 0,
        "ShortMessage": "",
        "VerificationType": 1,
        "OverdueRemind": 1,
        "OverdueRemind_Day": 3,
        "TimingOpen": 0,
        "TimingOpen_mode": 1,
        "TimingOpen_timegroup": {
            "Week1": "",
            "Week2": "",
            "Week3": "",
            "Week4": "",
            "Week5": "",
            "Week6": "",
            "Week7": ""
        },
        "TimingLocked": 0,
        "TimingLocked_timegroup": {
            "Week1": "",
            "Week2": "",
            "Week3": "",
            "Week4": "",
            "Week5": "",
            "Week6": "",
            "Week7": ""
        },
        "VisitorRootPassword": "",
        "MultiPerson": 1,
        "UseElevator": 0,
        "ElevatorPorts": [
            {
                "Num": 1,
                "RelayType": 2,
                "ReleaseTime": 2,
                "TimingOpen": {
                    "Use": 0,
                    "Open": 3,
                    "Timegroup": {
                        "Week1": "",
                        "Week2": "",
                        "Week3": "",
                        "Week4": "",
                        "Week5": "",
                        "Week6": "",
                        "Week7": ""
                    }
```

```
                }
            },
            {
                "Num": 64,
                "RelayType": 2,
                "ReleaseTime": 2,
                "TimingOpen": {
                    "Use": 0,
                    "Open": 3,
                    "Timegroup": {
                        "Week1": "",
                        "Week7": ""
                    }
                }
            }
        ],
        "FaceIR": 1,
        "FaceIRThreshold": 5,
        "FaceDistance": 3,
        "FaceThreshold": 58,
        "FPComparison": 80,
        "FaceMask": 0,
        "FaceMaskThreshold": 65,
        "Holidays": [],
        "Language": 1,
        "SystemTime": 1718421632,
        "UseNTP": 1,
        "TimeZone": 8,
        "Volume": 6,
        "Voice": 1,
        "ConnectPassword": "",
        "UseWired": 1,
        "WiredDHCP": 0,
        "WiredIP": "192.168.1.103",
        "WiredIPMask": "255.255.255.0",
        "WiredGteway": "192.168.1.1",
        "WiredDNS": "192.168.1.1",
        "WiredMAC": "7E-87-16-C1-E6-51",
        "UseWIFI": 0,
        "WIFIDHCP": 0,
        "WIFIIP": "192.168.1.150",
        "WIFIIPMask": "255.255.255.0",
        "WIFIGteway": "192.168.1.1",
        "WIFIDNS": "192.168.1.1",
        "WIFIMAC": "34-7D-E4-2D-63-3B",
        "WIFIAPName": "",
        "WIFIAPPassword": "",
        "UseWebPage": 1,
        "HTTPPort": 80,
        "HTTPSPort": 443,
        "WebPageUseSSL": 1,
        "UseUDP": 1,
        "UDPPort": 8101,
        "UseTelnet": 1,
        "TelnetPort": 23,
```

```
        "UseRTSP": 1,
        "RTSPPort": 554,
        "RTSPUser": "admin",
        "RTSPPassword": "12345678",
        "UseTCPClient": 0,
        "UseUDPClient": 0,
        "ServerAddress": "47.92.31.75",
        "ServerPort": 9003,
        "KeepaliveTime": 30,
        "UseHTTPClient": 0,
        "HTTPClient_ProtocolType": 100,
        "HTTPClient_ServerAddr": "http://192.168.1.100",
        "HTTPClient_KeepaliveTime": 30,
        "HTTPClient_UseGZIP": 0,
        "UseMQTTClient": 0,
        "UseMQTTSSL": 0,
        "MQTTServerAddr": "192.168.1.100",
        "MQTTPort": 0,
        "MQTTLoginName": "",
        "MQTTLoginPassword": "",
        "MQTTPublishTopic": "",
        "MQTTSubscribeTopic": "",
        "MQTT_KeepaliveTime": 30,
        "MQTT_UseGZIP": 0,
        "UseWebsocketClient": 0,
        "WebsocketClient_ProtocolType": 100,
        "WebsocketClient_ServerAddr": "ws://192.168.1.100/ws",
        "WebsocketClient_UseGZIP": 0,
        "WebsocketClient_KeepaliveTime": 30,
        "UseYZW": 0,
        "YZWAddr": "http://192.168.1.10",
        "RecordAutoCycle": 0,
        "SaveUnregistered": 1,
        "SaveRecordPicture": 1,
        "DeviceName": "",
        "Manufacturer": "",
        "ManufacturerPhone": "",
        "Website": "",
        "ProductionDate": "2024-06-15",
        "OEMText": "",
        "AutoRestart": 0,
        "AutoRestartTime": "02:00",
        "TimeGroups": [
            {
                "Num": 1,
                "Week1": "00:00-23:59",
                "Week2": "00:00-23:59",
                "Week3": "00:00-23:59",
                "Week4": "00:00-23:59",
                "Week5": "00:00-23:59",
                "Week6": "00:00-23:59",
                "Week7": "00:00-23:59"
            },
            {
                "Num": 64,
```

```
                "Week1": "",
                "Week7": ""
            }
        ],
        "DisplayBrightness": 6,
        "MenuPassword": "0000",
        "ShowIR": 0,
        "ShowPersonPhoto": 1,
        "PlayPersonName": 1,
        "RecognitionButon": 0,
        "UnregisteredWarn": 0,
        "ShowPersonName": 1,
        "FillLight": 2
}
```


Remote control

API-Remote operation command

Brief description:

- When the Remote value in the heartbeat packet return is 1, the device immediately requests this interface to obtain remote operation commands

Request URL:

- http://Server IP:port**/Device/RemoteCommand**

Request method:

- POST

Content-Type

- application/json

Request parameters:

Field	Type	Length	Required	Explain
SN	string	30	Yes	Device ID

Example of Request Parameters


```
{
    "SN":"FC-8200H12345678"
}
```

Return value parameter description

Parameter Name	Type	Necessary	Description
Success	int	Yes	1-- indicates success Other are error codes, and the error content needs to be indicated in the Message; 0--indicates failure
Message	string	No	Error message description, please refer to Chapter 3 for details
Restart	int	No	Remote restart: 0: no restart, 1: restart
Recover	int	No	Factory reset``0:Normal, 1: Factory reset
Opendoor	int	No	Remote door opening command` 0:Not processed, 1--Turn on the relay; 2--Keep the door normally opened; 3--Close the door (delete normally open); `4--Lock the door;5--Unlock the door
Closealarm	int	No	Close alarm command``0:Not processed, 1:Close all ongoing alarms and record them
RepostRecord	int	No	Re upload record``0:Not processed，1:Mark all uploaded records as not uploaded and resend them
PushAllPeople	int	No	Request to upload all stored personnel lists to the server At this point, the device calls the API [/People/PushPeople] to send the personnel list 0--Not processed，1--All personnel need to be uploaded.
QueryPeople	[uint]	No	Request to upload personnel with the specified user ID to the server. At this point, the device calls the API [/People/PushPeople] to send a list of personnel The type is an array
ClearRecord	int	No	Delete all records; 0-- Not processed 1--Delete all records
RegisterIdentifyTicket	object	No	Register the voucher type on the device For the object structure, please check the ** RegisterIdentifyTicket object structure in the ** RegisterIdentifyTicket command **After registration, call the API** /Device/RegisterIdentifyTicketResult** upload the results

Parameter Name	Type	Necessary	Description
PushSoftware	object	No	The download URL address for the firmware upgrade package of the push software After receiving the address, device starts downloading the upgrade package and will automatically update the firmware after successful verification Please refer to the firmware file object structure in the "PushSoftware command for the object structure**
PushSystemFile	[object]	No	Push system file update information, The data is of type array Please refer to the system file object structure in the ** PushSystemFile command for the object structure of each array element**
Snapshoot	int	No	Obtain device camera snapshot; 0-- Not processed 1--Camera snapshot and upload Upload snapshot and call API **/Device/RegisterIdentifyTicketResult**
OpenElevatorPort	string	No	Remote drive of elevator port comma separated string:1,2,3,4,5
KeepOpenElevatorPort	string	No	Put the elevator port into a normally open state comma separated string:1,2,3,4,5
CloseElevatorPort	string	No	Exit the elevator port from the normally open state comma separated string:1,2,3,4,5
LockElevatorPort	string	No	Put the elevator port into a locked state, and the user cannot drive it when locked comma separated string:1,2,3,4,5
UnlockElevatorPort	string	No	Exit the lock state of the elevator port comma separated string:1,2,3,4,5

RegisterIdentifyTicket Registration certificate object structure in command

| Field | Type | Required | Description |

| :---------- | :--- | :--- | ---------- |
| RegisterType | uint | Yes | Registered voucher type
View table **RegisterType Registration Type** |
| UserID | uint | No | User ID for registering credentials on the device |
| RegisterIndex | uint | No | Registered voucher index number
The index number for fingerprints is 0-9
The index number for the palm vein is 1-2|

RegisterType Registration Type

Value	Description
1	Register Card
2	Register Password
3	Register Fingerprint
4	Register Face
5	Register Palm Vein

PushSoftware The object structure of firmware files in commands

Field	Type	Length	Required	Description
SoftwareURL	string	1024	Yes	The URL address of the file
SoftwareMD5	string	64	Yes	MD5 hex encoding format for files with capital letters
SoftwareVer	string		Yes	The firmware version number of the file

PushSystemFile System file object structure in commands

Field	Type	Length	Required	Description
FileURL	string	1024	Yes	The URL address of the file
FileMD5	string	64	Yes	D5 hex encoding format for files with capital letters
FileType	int	1	Yes	File type Please refer to the file type definition for details **FileType File type**
FileIndex	int	1	Yes	File index
IsDelete	int	1	Yes	Does it indicate the need to delete files

FileType File Type

Value	Description
1	The standby image can have 8 images Support index range 1-8
2	There is only one startup image

Return value example

```
//Remote door opening
{
    "Success":1,
    "Opendoor":1
}

//Query designated personnel
{
    "Success":1,
    "QueryPeople":[1,2,3,4,5,6]
}

//Registration certificate  Register the face for the person with user ID 1
{
    "Success":1,
    "RegisterIdentifyTicket": 4,
    "RegisterIdentifyUserID": 1
}
```

API-Feedback on the results of personnel registration credentials

Brief description:

- After receiving the remote command of personnel registration certificate, the device enters the registration certificate interface and waits for the operation result. After the registration certificate is completed, call this API to return the registration result of personnel certificate
- And call the push personnel API to push the latest information of registered personnel
- Using the form upload method, Content-Type : multipart/form-data, the returned result may contain images

Request URL:

- http://Server IP:port**/Device/RegisterIdentifyTicketResult**

Request method:

- POST

Content-Type

- multipart/form-data

form-data Field

Field	Type	Length	Required	Description
ResultJson	string		Yes	Registration result JSON string
Photo	file		No	When personnel include photos, return photos

ResultJson Structure:

Field	Type	Length	Required	Description
SN	string	30	Yes	Device ID
UserID	uint		Yes	User Number
Result	int		Yes	Registration certificate result 1,Successful registration; 2,User cancels operation; 3--Duplicate registration information 4--No support fingerprint 5--No support palm vein 6--Device busy
RepetitionUserID	uint		No	When the registered credentials are duplicated, the duplicate user ID will be displayed here
UserDetail	object		No	After successful registration, the latest details of the personnel will be returned. If there are personnel photos they will be returned

Example of Request Parameters

```
//Repeat registration return
{
    "SN":"FC-8200H12345678",
    "UserID": 1,
    "Result": 2,
    "RepetitionUserID": 2
}
//Successful registration returned
{
    "SN":"FC-8200H12345678",
    "UserID": 1,
    "Result": 1,
    "UserDetail": {
        "UserID": "3",
        "Name": "888888",
        "Job": "Development Department",
        "Department": "Sales Department",
        "IdentityCard": "",
        "Attachment": "",
        "Photo": "http://abc.com/photo/1.jpg",
        "PhotoMD5": "613D870CA99EDF074BEE4387BAB09070",
        "PhotoLen": 55020,
        "Password": "2222",
        "CardNum": "6666",
        "AccessType": 0,
        "ExpirationDate": 0,
        "OpenTimes": 65535,
```

```
        "KeepOpen": 1,
        "Timegroup": 6,
        "Holidays": "1,3,9,10,11,17,21,25,27,30",
        "Elevators": "1,2,3,4,5",
        "FaceFeature": "http://abc.com/face/1.dat",
        "FaceFeatureMD5": "613D870CA99EDF074BEE4387BAB09070",
        "Fingerprints": [
            {
                Num: 1,
                Data: "abcdefgrtygnfhgfjhk ... jhkghjkgj==",
                MD5: "abcdefg"
            }
        ],
        "Palmveins": [
            {
                Num: 1,
                Data: "abcdefgrtygnfhgfjhk ... jhkghjkgj==",
                MD5: "abcdefg"
            }
        ]
    }
}
```

Parameter description of Return value

Parameter Name	Type	Required	Description
Success	int	Yes	1--indicates success Other are error codes, and the error content needs to be indicated in the Message; 0--indicates failure
Message	string	No	Error message description, please refer to Chapter 3 for details

Return value example

```
{
    "Success": 1
}
```

API-Upload device camera snapshot

Brief description:

- When receiving a remote command to obtain a camera snapshot, the device immediately takes a snapshot of the camera and calls this interface to upload it
- Using the form upload method, Content-Type : multipart/form-data, The returned result includes images

Request URL:

- http://Server IP:port**/Device/UploadSnapshoot**

Request method:

- POST

Content-Type

- multipart/form-data

form-data Field

Field	Type	Length	Required	Description
SN	string		Yes	Device SN
Photo	file		No	Camera snapshot of the device

Parameter description of return value

Parameter Name	Type	Necessary	Description
Success	int	Yes	1--indicates success Other are error codes, and the error content needs to be indicated in the Message; 0--indicates failure
Message	string	No	Error message description, please refer to Chapter 3 for details

Return value example

```
{
    "Success": 1
}
```

#Personnel management

API-Obtain Personnel Authorization Information

Brief description:

--When the heartbeat is kept alive and the server return value with a field for obtaining personnel and this value is 1, the device immediately initiates this request and repeats the request

-Stop request condition: 1. The server returns Success:1, but the personnel list is empty; 2. Success==0

Request URL:

- http://Server IP:port**/People/DownloadPeopleList**

Request Method:

- POST

Content-Type

- application/json

Request parameters

Field	Type	Length	Required	Description
SN	string	30	Yes	Device ID
Limit	int	10	Yes	The maximum number of personnel returned on each request is 1000, and the device sets this value based on its own processing capacity

Example of Request Parameters

```
{
    "SN":"FC-8200H12345678",
    "Limit": 100
}
```

Return value parameter description

Parameter Name	Type	Necessary	Description
Success	int	Yes	1--indicates success Other are error codes, and the error content needs to be indicated in the Message; 0: Success
Message	string	No	Error message description, please refer to Chapter 3 for details
PeopleCount	int	Yes	The number of personnel returned this time
PeopleList	Array	Yes	An array containing personnel permission information structure

PeopleJson Personnel data format

Parameter Name	Type	Neccessary	Description
UserID	string	Yes	User ID (maximum number 4294967295 type UINT32)
Name	string	No	Personnel name (characters<64 bits)
Job	string	No	Position (characters<64 bits)

Parameter Name	Type	Neccessary	Description
Department	string	No	Department (characters<64 bits)
IdentityCard	string	No	Identification card number (can be blank) (characters<64 digits)
Attachment	string	No	Other personnel information (characters<200)
Photo	string		Photo information, starting with http or https means using a URL address, otherwise it means using base64
PhotoMD5	string	No	MD5 HEX string format for photos (characters=32 digits)
PhotoLen	int	No	The maximum length of the photo supported is 400KB
Password	string	No	Password, only number, length: (0/4-8)
CardNum	string	No	Card number (digits, maximum value 1844674407909551615 type UINT62)
QRCode	string	No	Personnel QR code information (characters<128)
AccessType	int	No	Role 0--Ordinary personnel; 1--Administrator; 2-- Blacklist
ExpirationDate	uint32	No	Permission expiration date Unix timestamp in seconds 0 indicates an indefinite period Maximum indicates December 31, 2099
OpenTimes	int	No	Door opening times 0-65535; 65535-- indicates no restrictions, 0--indicates no passage allowed
KeepOpen	int	No	Normally opened card?, 1--Yes;0--No
Timegroup	int	No	Opening time zone group 1-64; 0- indicates restricted
Holidays	string	No	Holiday restrictions comma separated: 1, 2, 3, 4, 5
Elevators	string	No	Elevator port permission array comma separated: 1, 2, 3, 4, 5

Parameter Name	Type	Neccessary	Description
FaceFeature	string	No	Facial feature codes starting with http or https indicate the use of a URL address, otherwise they indicate the use of base64. Download the feature code from a file containing the content of base64
FaceFeatureMD5	string	No	MD5 value HEX string format of facial feature code
Fingerprints	[]	No	Fingerprint object
Palmveins	[]	No	Palm print object

Characters refer to bytes, with one Chinese character occupying 3-4 bytes (encoded in UTF-8) and one English character occupying 1 byte

Holidays Holidays

- An empty string or no field indicates no holiday restrictions
- For specific restricted holiday numbers, comma separated: 1, 2, 3, 4, 5
- *Number indicates that it is subject to all holiday restrictions

- **Fingerprint Fingerprint Object**

Serial Number	Type	Required	Description
Num	int	Yes	Fingerprint index number
Data	string	Yes	The fingerprint signature code starting with http or https indicates the use of a URL address, otherwise it indicates the use of base64. Download the signature code from a file, and the content of the downloaded file is based on base64
MD5	string	No	MD5 value of feature code in HEX string format

```
[ //Structural Example
    {
        Num: 1,
        Data: "http://abc.com/fp/1.dat",
        MD5: "abcdefg"
    },
    {
        Num: 2,
        Data: "abcdefgrtygnfhgfjhk ... jhkghjkgj==",
        MD5: "abcdefg"
    }
]
```

- **Palmvein palm print object**

Serial Number	Type	Required	Description
Num	int	Yes	Palm print index number
Data	string	Yes	The palm print feature code starting with http or https indicates the use of a URL address, otherwise it indicates the use of base64. Download the feature code from a file, and the content of the downloaded file is based on base64
MD5	string	No	MD5 value of feature code in HEX string format

```
[ //Structural Example
    {
        Num: 1,
        Data: "http://abc.com/palm/1.dat",
        MD5: "abcdefg"
    },
    {
        Num: 2,
        Data: "abcdefgrtygnfhgfjhk ... jhkghjkgj==",
        MD5: "abcdefg"
    }
]
```

- **Elevator Elevator Authority**

```
 //Represents this person only has access permission on the 1st-5th floor of
elevator
 [
     1,2,3,4,5
 ]
 //Represents this person does not have elevator permission
 [

 ]
 //Represents this person only has access permission on the 10th floor of
elevator
 [
     10
 ]
```

Return parameter example


```
{
    "Success": 1,
    "Count": 1,
    "PeopleList": [
        {
            "UserID": "3",
            "Name": "888888",
            "Job": "Development",
            "Department": "Sales Department",
            "IdentityCard": "",
            "Attachment": "",
            "Photo": "http://abc.com/photo/1.jpg",
            "PhotoMD5": "613D870CA99EDF074BEE4387BAB09070",
            "PhotoLen": 55020,
            "Password": "2222",
            "CardNum": "6666",
            "AccessType": 0,
            "ExpirationDate": 0,
            "OpenTimes": 65535,
            "KeepOpen": 1,
            "Timegroup": 6,
            "Holidays": "1,3,9,10,11,17,21,25,27,30",
            "Elevators": "1,2,3,4,5",
            "FaceFeature": "http://abc.com/face/1.dat",
            "FaceFeatureMD5": "613D870CA99EDF074BEE4387BAB09070",
            "Fingerprints": [
                {
                    Num: 1,
                    Data: "http://abc.com/fp/1.dat",
                    MD5: "abcdefg"
                }
            ],
            "Palmveins": [
                {
                    Num: 1,
                    Data: "http://abc.com/palm/1.dat",
                    MD5: "abcdefg"
                }
            ]
        },
        {
            "UserID": "444",
            "Name": "444",
            ...
            "Fingerprints": [],
            "Palmveins": []
        }
    ]
}
```


API-Feedback to obtain personnel storage results

Brief description:

- After getting a batch of people from the server to import, call this interface to return the import results

Request URL:

- http://Server IP:port**/People/DownloadPeopleListResult**

Request method:

- POST

Content-Type

- application/json

Request parameters:

Field	Type	Length	Required	Description
SN	string	30	Yes	Device ID
SuccessCount	int	20	Yes	Number of successful imports
FailCount	int	20	Yes	Number of failed imports
FailList	[]		Yes	Reason code for import failure (see error code for specific description)

The structure of failure information in the array 'fail Employee Id'

Parameter Name	Type	Necessary	Description
UserID	string	Yes	User ID (maximum number 4294967295 type UINT32)
ErrorCode	int	Yes	Error Code
RepeatID	string	Yes	Duplicate user ID, this field indicates this person is repeating with which person
ErrMsg	string	No	Error Description

Example of Request Parameters

```
{
    "SN":"FC-8200H12345678",
    "SuccessCount":16,
    "FailCount":1,
    "FailCount":[
        {
            "UserID":xxx,
            "ErrorCode":20,
            "RepeatID":123,
             "ErrMsg" : ""
```

```
        },{
            "UserID":”xxx”,
            "UserID":1,
             "ErrMsg" : ""
    ]
}
```

*errorCode Personnel failure error code**

Success	Description
1	Abnormal personnel information parameters, specific reasons refer to
2	Personnel photo URL download failed
3	The personnel photo image is too large and does not meet the requirements. It needs to be less than 500kb
4	Facial feature code download failed
5	Fingerprint signature download failed
6	Palm vein signature download failed
20	Duplicate facial feature codes
21	Duplicate fingerprint feature codes
22	Duplicate palm vein feature codes
23	Duplicate card number
24	Personnel are full
25	An error occurred while querying data
26	Error occurred while saving personnel photos
27	Facial feature code format error
28	Fingerprint feature code format error
29	Format error of palm vein feature code
30	Photo not recognizable
31	Error occurred while saving fingerprint feature code
32	Error occurred while saving palm vein feature code

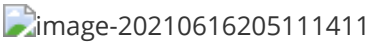
Return value parameter description

Parameter Name	Type	Necessary	Description

Parameter Name	Type	Necessary	Description
Success	int	Yes	1--indicates success Other are error codes, and the error content needs to be indicated in the Message; 0: Success
Message	string	No	Error message description, please refer to Chapter 3 for details

Return value example

```
{
    "Success":1
}
```



##API - Obtain the personnel to be deleted**

Brief description:

- After sending the heartbeat packet, the server calls back when there are personnel that need to be deleted, and cycle callback
- Stop cycle condition 1, Success:1,and deleteInfo is an empty array or does not have this field 2, Success: 0

Request URL:

- http://Server IP:port**/People/DeletePeopleList**

Request method:

- POST

Content-Type

- application/json

Request parameters:

Field	Type	Length	Required	Explain
SN	string	30	Yes	Device ID
Limit	int	10	Yes	Limit on the number of personnel which returned on each request (default 50 if not carrying, maximum 1000)

Example of Request Parameters

```
{
    "SN":"FC-8200H12345678"
}
```

Return value parameter description

Parameter Name	Type	Necessary	Description
Success	int	Yes	1--indicates success Other are error codes, and the error content needs to be indicated in the Message; 0: Success
Message	string	No	Error message description, please refer to Chapter 3 for details
DeleteAll	int	No	1: Clear all personnel information 0: Delete by specified user number
DeleteCount	int	Yes	Number of personnel to be deleted
DeleteList	[]	No	When DeleteAll=0, this parameter contains the user ID that needs to be deleted

Return value example

```
{
    "Success":1,
    "DeleteAll":0,
    "DeleteList":[
        1,2,3,4,5
    ]
}
```

API-Feedback on the operation results of deleting personnel

Brief description:

- After getting a batch of people from the server to import, call this interface to return the import results

Request URL:

Request method:

- POST

Content-Type

- application/json

Request parameters:

Field	Type	Length	Required	Explain
SN	string	30	Yes	Device ID

Field	Type	Length	Required	Explain
DeleteCount	int	20	Yes	Delete Quantity
DeleteAll	int	20	Yes	1 All personnel have been deleted；0 Only delete specified personnel
DeleteList	[]		Yes	Deleted personnel number

Example of Request Parameters

```
{
    "SN":"FC-8200H12345678",
    "DeleteCount":5,
    "DeleteAll":0,
    "DeletList":[1,2,3,4,5]
}
```

Parameter Name	Type	Neccessary	Description
Success	int	Yes	1--indicates success Other are error codes, and the error content needs to be indicated in the Message ; 0: Success
Message	string	No	Error message description, please refer to Chapter 3 for details

Return value example

```
{
    "Success":1
}
```

API-Push Personnel Information

Brief description:

- When personnel are added or modified on the device, the changed information will be uploaded to the cloud platform
- When the server requests to upload specified personnel information, push the specified personnel information to the platform
- When the server requests to upload all personnel information, push all personnel information to the platform

Request URL:

- http://Server IP:port**/People/PushPeople**

Request method:

- POST

Content-Type

- multipart/form-data

Request Paramters:

Field	Type	Length	Required	Explain
SN	string	30	Yes	Device ID
PushType	int	1	Yes	Types of personnel changes in device 1--Add New；2--Update；3--Delete；4--Query；
Detail	class		No	Personnel details, if personnel do not exist, this field is not available
Photo	file		No	When there are personnel photos, photo files will be uploaded
UserID	uint32		Yes	Personnel ID The personnel ID for this push

Request parameter examples with personnel photos, fingerprint feature codes, and palm print feature codes

```
POST /note/insertNoteFace HTTP/1.1
Accept: */*
Host: localhost:5000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: multipart/form-data; boundary=--------------------------
-506873351428002157394455
Content-Length: 17842

----------------------------506873351428002157394455
Content-Disposition: form-data; name="SN"

FC-8200H12345678
----------------------------506873351428002157394455
Content-Disposition: form-data; name="PushType"

1
----------------------------506873351428002157394455
Content-Disposition: form-data; name="Detail"
Content-Encoding: gzip
//The following content needs to be compressed with gzip before transmission.
JSON formatted website https://www.json.cn/
        {
            "UserID": "3",
            "Name": "888888",
            "Job": "Development",
```

```
            "Department": "Sales Department",
            "IdentityCard": "",
            "Attachment": "",
            "Photo": "http://abc.com/photo/1.jpg",
            "PhotoMD5": "613D87OCA99EDF074BEE4387BAB09070",
            "PhotoLen": 55020,
            "Password": "2222",
            "CardNum": "6666",
            "AccessType": 0,
            "ExpirationDate": 0,
            "OpenTimes": 65535,
            "KeepOpen": 1,
            "Timegroup": 6,
            "Holidays": "1,3,9,10,11,17,21,25,27,30",
            "Elevators": "1,2,3,4,5",
            "FaceFeature": "http://abc.com/face/1.dat",
            "FaceFeatureMD5": "613D87OCA99EDF074BEE4387BAB09070",
            "Fingerprints": [
                {
                    Num: 1,
                    Data: "abcdefgrtygnfhgfjhk ... jhkghjkgj==",
                    MD5: "abcdefg"
                }
            ],
            "Palmveins": [
                {
                    Num: 1,
                    Data: "abcdefgrtygnfhgfjhk ... jhkghjkgj==",
                    MD5: "abcdefg"
                }
            ]
        }
----------------------------506873351428002157394455
Content-Disposition: form-data; name="Photo"; filename="Photo.jpg"
Content-Type: image/jpeg

*****jpeg file binary content*****
----------------------------506873351428002157394455--
```

Return value parameter description

Parameter Name	Type	Neccessary	Description
Success	int	Yes	1-- indicates success Other are error codes, and the error content needs to be indicated in the Message; 0: Success
Message	string	No	Error message description, please refer to Chapter 3 for details

Return value example

```
{
    "Success":0
}
```

Online Authentication

API-Request server authentication

Brief description:

- Use this interface when there are new punch-in records in the device
  -This interface only supports uploading one check-in record at a time

Request URL:

- http://Server IP:port**/Device/RequestAuthorization**

Request method:

- POST

Content-Type

- multipart/form-data

Parameters:

Field	Type	Length	Neccessary	Description
SN	string	30	Yes	Device SN
RecordDetail	string	1000	Yes	Record details, JSON string
Photo	file	50KB	No	This parameter is only required when there are images in the record

RecordDetail Field Description

Field	Type	Length	Neccessary	Description
RecordID	long	10	Yes	Request ID
RecordType	int	5	Yes	Event type
RecordDate	int64	20	Yes	Punch-in time unix timestamp
UserID	string	20	No	User ID
Code	string	64	No	Personnel ID
Name	string	64	No	Personnel Name
IdentityCard	string	64	No	Identification
Job	string	64	No	Position

Field	Type	Length	Neccessary	Description
Department	string	64	No	Department
CardNum	string	20	No	Card Number
QRCode	string	128	No	QR Code
IsEntry	int	1	No	Is it entry? 1 indicates entry, 0 indicates exit
BodyTemp	int	5	No	Human body temperature measurement requires a division of 10
PhotoLen	int	10	No	Image file length 0 indicates no image

RecordDetail Parameter Examples

```
{
    "RecordID": 1718616771001,
    "RecordType":   3,
    "RecordDate":   1718616771,
    "UserID":   "1",
    "Name": "1",
    "IdentityCard": "",
    "Job":  "",
    "Department":   "",
    "CardNum":  "0",
    "QRCode":   "",
    "IsEntry":  1,
    "BodyTemp": 0,
    "PhotoLen": 40363
}
```

Parameter Examples

```
POST /Device/RequestAuthorization HTTP/1.1
Accept: */*
Host: localhost:5000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: multipart/form-data; boundary=--------------------------
-506873351428002157394455
Content-Length: 17842
----------------------------506873351428002157394455
Content-Disposition: form-data; name="SN"

 FC-838OT12345678
----------------------------506873351428002157394455
Content-Disposition: form-data; name="RecordDetail"
Content-Encoding: gzip
Content-Type: application/octet-stream
```

```
//When request compression is enabled, the following content needs to be
compressed by gzip before transmission when uploading.
{ "RecordID":120,"RecordType":3,"RecordDate":1718616771,
"UserID":"1","Name":"1","IdentityCard":"","Job":"","Department":"","CardNum":"0"
,"QRCode":"","IsEntry":1,"BodyTemp":0,"PhotoLen":40363}
----------------------------506873351428002157394455
Content-Disposition: form-data; name="pic"; filename="Postman_file.jpg"
Content-Type: image/jpeg

*****jpeg Binary content of file*****
----------------------------506873351428002157394455--
```

Return Example

```
{
    "Success": 1
}
```

Request content compression

- If the recordJson field in 'from data' is too long, the content compression option can be used to include Content-Encoding in the paragraph gzip

```
Content-Disposition: form-data; name="RecordDetail"
Content-Encoding: gzip

****Compressed binary content****
```

Return parameter description

Parameter Name	Type	Necessary	Description
Success	int	Yes	Please refer to the table for**Success returns the results**
Message	string	No	When door opening is prohibited, the server shall explain the reason for the prohibition and the device shall display it on the screen to the check-in personnel.

Success returns results

Value	Description
0	Authentication failed, refused to open the door
1	Authentication successful, open the door and display authentication message
2	Authentication successful, open the door, no message displayed

Record management

API-Upload punch-in records

Brief description:

- Use this interface when there are new punch-in records in the device
- This interface only supports uploading one punch-in record at a time

Request URL:

- http://Server IP:port**/Record/UploadIdentifyRecord**

Request method:

- POST

Content-Type

- multipart/form-data

Parameters:

Field	Type	Length	Required	Explain
SN	string	30	Yes	Device SN
RecordDetail	string	1000	Yes	Record details, JSON string
Photo	file	50KB	No	This parameter is only required when there are images in the record

RecordDetail Field Description

Field	Type	Length	Required	Explain
RecordID	long	10	Yes	Record serial number
RecordType	int	5	Yes	Event type
RecordDate	int64	20	Yes	Punch in time Unix timestamp
UserID	string	20	No	User ID
Name	string	30	No	Personnel Name
IdentityCard	string	5	No	Identity card
Job	string	1	No	Position
Department	string	1	No	Department
CardNum	string	20	No	Card No.

Field	Type	Length	Required	Explain
QRCode	string	128	No	QR Code
IsEntry	int	1	No	Is it entry? 1 indicates entry, 0 indicates exit
BodyTemp	int	5	No	Human body temperature measurement needs to be divided by 10
PhotoLen	int	10	No	Image file length 0 indicates no image

RecordType Event type

Value	Explain
1	Card Verification
2	Fingerprint Verification
3	Facial Verification
4	Card + Fingerprint
5	Face + Fingerprint
6	Card + Face
7	Card + Password
8	Face + Password
9	Fingerprint + Password
10	Password verification user number and password
11	Card + Fingerprint + Password
12	Card+ Face + Password
13	Fingerprint + Face + Password
14	Card + Fingerprint + Face
15	Repeated Verification
16	Expired Validity Period
17	Opening Time Zone Expired
18	Cannot open the door during holidays
19	Unregistered User
20	Detected Locked

Value	Explain
21	The number of valid times has been exhausted
22	Verification while locked, prohibit opening the door
23	Lost Reported Card
24	Blacklist Card
25	Open the door without verification -- when pressing the fingerprint, the user number is 0, and when swiping the card, the user number is the card number
26	Prohibit card swiping verification -- When card swiping is disabled in the [Permission Authentication Method]
27	Prohibit fingerprint verification -- [Permission authentication method] When fingerprint is disabled
28	Controller Expired
29	Verified Passed - Validity period is ready to expire
30	Abnormal body temperature, refusal to enter
31	Visitor password to open the door
32	Scanning dynamic QR code to open the door
33	Add user to the device menu
34	Modify user in the device menu
35	Delete user from the device menu
36	Palm Print Recognition
37	Card + Palm Print+ Face
38	Palm Print + Password
39	Card + Palm Print
40	Face + Palm Print
41	Card + Palm Print + Password
42	Palm Print + Face + Password
43	Fingerprint + Palm Print+ Face
44	Combination verification --waiting for the next person
45	Combination Verification Failed
46	Combination Verification Successful
47	People and ID Comparison

Value	Explain
48	Card Not Registered
49	Unregistered QR Code

RecordDetail Parameter examples

```
{
    "RecordID": 120,
    "RecordType":   3,
    "RecordDate":   1718616771,
    "UserID":   "1",
    "Name": "1",
    "IdentityCard": "",
    "Job":  "",
    "Department":   "",
    "CardNum":  "0",
    "QRCode":   "",
    "IsEntry":  1,
    "BodyTemp": 0,
    "PhotoLen": 40363
}
```

** Request Example**

```
POST /note/insertNoteFace HTTP/1.1
Accept: */*
Host: localhost:5000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: multipart/form-data; boundary=--------------------------
-506873351428002157394455
Content-Length: 17842
----------------------------506873351428002157394455
Content-Disposition: form-data; name="SN"

 FC-838OT12345678
----------------------------506873351428002157394455
Content-Disposition: form-data; name="recordJson"
Content-Encoding: gzip
Content-Type: application/octet-stream

//When request compression is enabled, the following content needs to be
compressed by gzip before transmission when uploading.
{ "RecordID":120,"RecordType":3,"RecordDate":1718616771,
"UserID":"1","Name":"1","IdentityCard":"","Job":"","Department":"","CardNum":"0"
,"QRCode":"","IsEntry":1,"BodyTemp":0,"PhotoLen":40363}
----------------------------506873351428002157394455
Content-Disposition: form-data; name="pic"; filename="Postman_file.jpg"
Content-Type: image/jpeg

*****jpeg file binary content*****
```

```
----------------------------506873351428002157394455--
```

####* **Return Example**

```
{
    "Success": 1
}
```

Return error code

```
{
    "Success":401   //Returning 401 indicates that the device is not authorized and will no longer send punch in records. The device parameters will be sent to the server. But it will still periodically send keep alive packages
}
//The device returns 401, indicating that the record was not successfully uploaded
```

- The device waits for a server response time of 30 seconds;
- If the server returns an error code, wait for 10 seconds and then re-upload the record;

Request content compression

- If the recordJson field in 'fromdata' is too long, you can use the content compression option and include Content Encoding: gzip in the paragraph

  ```
  Content-Disposition: form-data; name="recordJson"
  Content-Encoding: gzip

    ****Compressed binary content****
  ```

  Return Parameter Description

Parameter Name	Type	Necessary	Description
Success	int	Yes	1-- indicates success Other are error codes, and the error content needs to be indicated in the Message; 0: Success

API-Upload System Records

Brief description:

- Use this interface when there are new system records in the device
- This interface will batch upload system records

Request URL:

- http://Server IP:port**/Record/UploadSystemRecord**

Request method:

- POST

Content-Type

- application/json

Parameter:

Field	Type	Length	Required	Explain
SN	string	30	Yes	Device SN
RecordType	int	1	Yes	Record Type
Records	【】	1000	Yes	Record List

Record Field Description

Field	Type	Length	Required	Explain
RecordID	long	10	Yes	Record number
RecordType	int	5	Yes	Event type 1- Door Sensor Record; 2-- System Record
RecordDate	int64	20	Yes	Punch in time Unix timestamp

RecordType Field Description

Value	Explain
1	Door Sensor Record
2	System Record

Value	Explain
1	Door Sensor-Open Door
2	Door Sensor-Close Door
3	Enter the door sensor alarm detection state
4	Exit the door sensor alarm detection state
5	The door is not closed properly
6	Use the button to open the door
7	When the button is pressed to open the door, the door is already locked

Value	Explain
8	The controller has expired when use the button to open the door

Value	Explain
1	Software Open Door
2	Software Close Door
3	Software Normally Opened
4	The controller automatically enters normally open mode
5	The controller automatically close the door
6	Press and hold the exit button to enters normally opened mode
7	Press and hold the exit button to enters normally closed mode
8	Software Locked
9	Software Removed Locked
10	Controller timing locked--automatic locking at specified time
11	Controller timing locked--automatic remove locking at specified time
12	Alarm--Locked
13	Alarm--Removed Locked
14	Illegal authentication Alarm
15	Door Sensor Alarm
16	Forced Alarm
17	Opening Time Out Alarm
18	Blacklist Alarm
19	Fire Alarm
20	Tamper Alarm
21	Illegal Authentication Alarm Removed
22	Door Sensor Alarm Removed
23	Forced Alarm Removed
24	Timing Opening Timeout Alarm Removed
25	Blacklist Alarm Removed

Value	Explain
26	Fire Alarm Removed
27	Tamper Alarm Removed
28	System Powered On
29	System Error Reset (Watchdog)
30	Device Formatting Record
31	Card Reader Reversed Connection
32	The card reader circuit is not properly connected
33	Unrecognized card reader
34	The network cable has been disconnected
35	The network cable has been inserted
36	WIFI Connected
37	WIFI Disconnected
38	Bluetooth Door Opening
39	Call the Roll Timeout
40	Clear all personnel from the device menu
41	Backup personnel to USB drive in the device menu
42	Import personnel from USB drive in the device menu
43	Remote door opening for indoor unit
44	Delete all records
45	Delete all personnel

RecordDetail Parameter examples

```
[
    {
        "RecordID":    1,
        "RecordType":  1,
        "RecordDate":  1718616771
    },
    {
        "RecordID":    2,
        "RecordType":  1,
        "RecordDate":  1718616772
    }
]
```


Parameter examples


```
{
    "SN":  "FC-8380T12345678",
    "RecordType":  1,
    "Records":  [
        {
            "RecordID":    1,
            "RecordType":  1,
            "RecordDate":  1718616771
        },
        {
            "RecordID":    2,
            "RecordType":  1,
            "RecordDate":  1718616772
        }
    ]
}
```


Return Example


```
{
    "Success": 1
}
```


Return error code

```
{
    "Success":401   //Returning 401 indicates that the device is not authorized and will no longer send punch in records. The device parameters will be sent to the server. But it will still periodically send keep alive packages
}
//The device returns 401, indicating that the record was not successfully uploaded this time
```

- The device waits for a server response time of 30 seconds;
- If the server returns an error code, wait for 10 seconds and then re-upload the record;

Return parameter description

Parameter Name	Type	Necessary	description
Success	int	Yes	1- indicates success Other are error codes, and the error content needs to be indicated in the Message; 0: Success